

Reproduction réduite de DINO

Emerging Properties in Self-Supervised Vision Transformers
sur STL-10 avec un ViT-Tiny

Namie Pajot Alexandra Arucan Mael Zapata Philipp Widenfels
Albert School — Bachelor Business & Data (3^e année)
Cours : Introduction to Deep Learning — A. Bouru-Gazeau

9 juin 2026

Résumé

Nous reproduisons, dans un cadre volontairement réduit mais rigoureux, l'idée centrale de **DINO** [1] : l'apprentissage auto-supervisé de représentations visuelles par *self-distillation* avec un réseau enseignant construit comme moyenne mobile exponentielle (EMA) de l'étudiant, sans aucune étiquette. Nous entraînons un Vision Transformer *ViT-Tiny* (patch 8, $\sim 5\text{--}6\text{M}$ paramètres) sur le jeu de données STL-10, évaluons la qualité des représentations gelées par un classifieur k -NN, et comparons à deux lignes de base de même architecture (initialisation aléatoire et entraînement supervisé). Nous reproduisons qualitativement la propriété émergente phare du papier — des cartes d'auto-attention qui segmentent les objets sans supervision — et menons deux ablations exécutées inspirées de la Table 7 du papier (encodeur à momentum, multi-crop) ; une troisième (température de l'enseignant) est décrite mais non exécutée faute de budget. Nous comparons nos *effets relatifs* d'ablation à ceux du papier plutôt que les valeurs absolues. Notre but n'est pas d'égaler les performances ImageNet du papier mais de montrer que le mécanisme fonctionne à échelle réduite et que nos ablations en retrouvent qualitativement les conclusions.

1 Introduction

Papier choisi. DINO — *Emerging Properties in Self-Supervised Vision Transformers*, Caron *et al.*, ICCV 2021 (arXiv :2104.14294) [1]. DINO (*self-distillation with **no** labels*) est une méthode d'apprentissage auto-supervisé (SSL) qui apprend des représentations visuelles sans étiquettes en faisant correspondre les sorties d'un réseau *étudiant* à celles d'un réseau *enseignant* construit à partir de l'étudiant lui-même.

Pourquoi ce papier. Trois raisons ont motivé ce choix. (i) *Pertinence* : le pré-entraînement auto-supervisé répond directement au coût d'annotation, goulot d'étranglement des pipelines réels ; DINOv2 (Meta, 2023) en est le descendant direct et largement utilisé en production. (ii) *Évaluabilité* : la contribution est à la fois quantitative (précision k -NN de représentations gelées) et qualitative (cartes d'attention qui segmentent les objets), ce qui donne un rapport plus riche. (iii) *Faisabilité* : l'algorithme tient en ~ 15 lignes de pseudo-code (Algorithme 1 du papier), ce qui rend une reproduction réduite réaliste sur Colab. Les alternatives envisagées (MAE, Barlow Twins, DDPM) ont été écartées respectivement pour un résultat visuel moins frappant à petite échelle, un espace d'ablation plus pauvre, et un coût de calcul incompatible avec Colab.

Données et angle d'analyse. Nous travaillons sur **STL-10**, choisi pour sa résolution 96×96 (adaptée à la tokenisation en patches d'un ViT) et son *split non étiqueté* de 100 000 images, conçu pour le SSL. Notre angle est double :

1. **quantitatif** — la représentation auto-supervisée est-elle utile sans fine-tuning ? Mesure par k -NN à features gelées, comparée à une ligne de base supervisée de même architecture et à une borne basse (initialisation aléatoire) ;
2. **qualitatif et mécaniste** — la propriété émergente (attention sémantique) se manifeste-t-elle à échelle réduite, et quels composants sont *indispensables* pour éviter l’effondrement (*collapse*) des représentations ? C’est l’objet de nos ablations.

2 Méthode

2.1 Vue d’ensemble de DINO

DINO entraîne deux réseaux de même architecture, un étudiant g_{θ_s} et un enseignant g_{θ_t} . Chaque est composé d’un *backbone* (ici un ViT) suivi d’une tête de projection produisant un vecteur de dimension K , transformé en distribution de probabilité par un softmax avec température. L’objectif est de rendre la distribution de l’étudiant cohérente avec celle de l’enseignant sur *des vues différentes de la même image*. La Figure 1 en résume l’architecture.

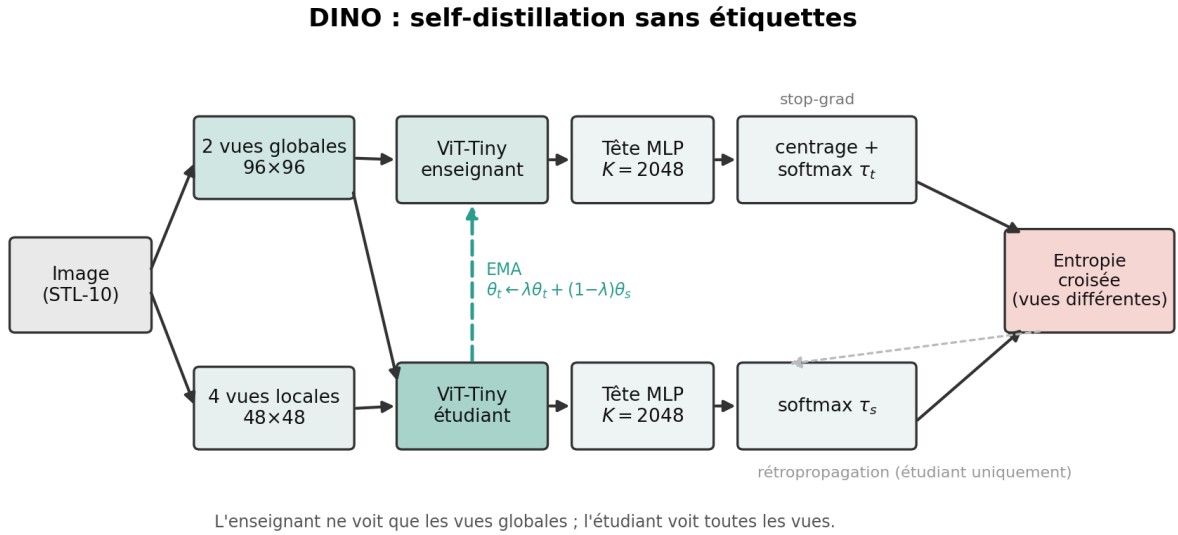


FIGURE 1 – Architecture de DINO. Une image produit 2 vues globales (96×96) et 4 vues locales (48×48). L’enseignant — moyenne mobile exponentielle (EMA) de l’étudiant, avec *stop-grad* — ne traite que les vues globales ; l’étudiant traite toutes les vues. Chaque branche enchaîne un ViT-Tiny puis une tête de projection ($K = 2048$) ; la perte est l’entropie croisée entre la distribution *enseignant* (centrée puis affûtée) et la distribution *étudiant*, sur les paires de vues différentes.

Multi-crop. À partir d’une image, on génère un ensemble de vues : 2 vues *globales* (grandes) et plusieurs vues *locales* (petites). L’enseignant ne voit que les vues globales ; l’étudiant voit *toutes* les vues. La perte n’est calculée que sur les paires de vues *différentes*, ce qui pousse l’étudiant à inférer le contexte global à partir d’un détail local (« local-to-global »).

Enseignant à momentum (EMA). L’enseignant n’est jamais entraîné par rétropropagation : ses poids suivent une moyenne mobile exponentielle de ceux de l’étudiant,

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s, \quad (1)$$

avec λ suivant un planning cosinus de 0,996 vers 1. L’enseignant joue ainsi le rôle d’une cible stable et légèrement « en avance », analogue à un ensemble de modèles (Polyak averaging).

Perte et anti-effondrement. Soit P_t et P_s les distributions de l’enseignant et de l’étudiant. La perte est une entropie croisée

$$\mathcal{L} = \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} -P_t(x) \log P_s(x'), \quad (2)$$

où V est l’ensemble des vues. Deux mécanismes *complémentaires* évitent l’effondrement des représentations (tout mapper vers un même vecteur) :

- **centrage** : on soustrait à la sortie de l’enseignant une moyenne c mise à jour par EMA sur le batch ($c \leftarrow m c + (1 - m) \overline{g_{\theta_t}}$). Le centrage empêche une dimension de dominer ;
- **affûtage** (*sharpening*) : la température de l’enseignant τ_t est basse (0,04), ce qui produit des cibles piquées. L’affûtage empêche au contraire l’effondrement vers la distribution uniforme. Centrage et affûtage agissent en sens opposés ; leur équilibre est crucial (objet de l’ablation A3).

2.2 Différences avec une ligne de base classique

Comparé à un entraînement supervisé standard, DINO se distingue par : (i) *absence d’étiquettes* — l’objectif est la cohérence inter-vues, pas la prédiction de classe ; (ii) *cible dynamique* — l’enseignant EMA remplace les étiquettes fixes ; (iii) *pas de paires négatives* ni de mémoire de contraste (contrairement à SimCLR/MoCo) et *pas de prédicteur* (contrairement à BYOL) ; (iv) *évaluation à features gelées* (k -NN), sans tête de classification entraînée. Ces choix rendent la méthode simple à implémenter tout en produisant des représentations transférables.

3 Cadre de reproduction

3.1 Ce qui est reproduit fidèlement

Le cœur de la méthode : architecture ViT + tête de projection, *multi-crop*, enseignant EMA, perte d’entropie croisée inter-vues, centrage + affûtage, planning cosinus du momentum et du taux d’apprentissage, règle de mise à l’échelle linéaire du LR ($\text{lr} = 0,0005 \times \text{bs}/256$), et l’évaluation k -NN à features gelées.

3.2 Jeu de données et prétraitements

STL-10 : split *unlabeled* (100 000 images 96×96) pour le pré-entraînement ; split *train* (5 000 images étiquetées) comme banque de référence k -NN et pour la ligne de base supervisée ; split *test* (8 000 images) pour l’évaluation. Augmentations : pour les vues globales, **RandomResizedCrop**(96, scale = [0,4,1]), flip horizontal, *color jitter*, niveaux de gris aléatoires, flou gaussien, puis normalisation par les moyennes/écarts-types de STL-10 ; pour les vues locales, recadrage 48×48 avec scale=[0,05,0,4]. L’évaluation utilise un simple **Resize/CenterCrop** à 96 sans augmentation.

3.3 Métriques et objets d’analyse

- **Précision k -NN top-1** ($k = 20$, similarité cosinus pondérée par température) sur les features gelées du backbone enseignant : mesure la qualité « brute » des représentations, sans entraînement supplémentaire.
- **Cartes d’auto-attention** du token [CLS] du dernier bloc ViT : preuve qualitative de l’émergence d’une attention sémantique.
- **Courbe k -NN sonde** de l’entraînement adaptatif (diagnostic de convergence et de saturation, Figure 6).

3.4 Ce qui est simplifié par rapport au papier

Le Tableau 1 résume les réductions. Elles sont *assumées* : l’objectif est de démontrer le mécanisme sous contrainte Colab, pas d’égaliser les chiffres ImageNet. Le notebook expose un drapeau `FAST_MODE` ; les réglages fidèles au papier restent disponibles (`FAST_MODE=False`).

TABLE 1 – Réglages du papier vs. notre reproduction réduite (mode `FAST_MODE`).

Élément	Papier (DINO)	Cette repro.
Jeu de données	ImageNet-1k (1,28 M)	STL-10 (adaptatif, $\leq 24\,000$)
Backbone	ViT-S/16, ViT-B/8	ViT-Tiny/8 ($\sim 5\text{--}6$ M)
Résolution	224	96 (global) / 48 (local)
Époques	300	~ 60 (adaptatif), 20 (ablations)
Taille de batch	1024	256
Dim. de sortie K	65 536	2 048
Vues (global+local)	2 + 8/10	2 + 4
Évaluation	lin. probing + k -NN	k -NN ($k=20$)

Justification des principaux choix. ViT-Tiny est la plus petite variante ViT standard et tient sur un GPU T4. Un patch de taille 8 sur des images 96 donne $12 \times 12 = 144$ tokens, suffisant pour une attention exploitable (un patch 16 n’en donnerait que 36). STL-10 est préféré à CIFAR-10/100 pour sa résolution *et* son split non étiqueté dédié au SSL. Le détail des arbitrages figure dans `decisions.md`.

3.5 Contraintes d’exécution : Colab T4 vs. Apple MPS

Le notebook visait initialement le GPU **T4** de Google Colab (`FAST_MODE` y est calibré pour $\sim 2\text{--}3$ h). Deux obstacles concrets nous ont fait migrer l’exécution :

- **Mémoire GPU.** Aux réglages par défaut (batch 256), la passe *multi-crop* — six vues par image (2 globales $96^2 + 4$ locales 48^2) traversant l’étudiant *et* l’enseignant, tête de dimension $K = 2048$ — déclenche une *OutOfMemoryError* sur le T4 (16 Go). On la contourne en divisant le batch par deux ($256 \rightarrow 128$) et en ajustant le LR par la règle de mise à l’échelle linéaire, au prix d’un réglage dégradé.
- **Coupures de session.** Le *runtime* gratuit se déconnecte sur inactivité ; combiné à un entraînement de plusieurs heures, cela rend peu fiable l’achèvement du pipeline avant l’échéance.

Nous avons donc basculé sur une machine locale **Apple Silicon (M3)** via le *backend MPS* de PyTorch, échangeant du débit brut contre une exécution *ininterrompue* (pas de coupure de session ; 36 Go de mémoire unifiée). Cela a exigé quelques adaptations propres à MPS :

- `dynamic_img_size=True` sur le ViT (timm), pour accepter les vues locales 48×48 : sans cela le *patch embed* (strict) rejette toute entrée $\neq 96$ et le multi-crop ne tourne pas ; les plongements de position sont alors ré-interpolés par taille de vue.
- L’opérateur `_upsample_bicubic2d_aa_backward` (rétropropagation de cette interpolation) n’est pas implémenté sur MPS et bascule sur CPU (léger surcoût) ; `pin_memory` et la précision mixte (AMP) sont également indisponibles, d’où un entraînement en `float32`.

Au final, le *pipeline* complet (pré-entraînement adaptatif + baseline supervisée + ablations A1/A2) a tourné sans interruption en ~ 6 h sur le GPU intégré, là où l’environnement T4 — même après réduction du batch — restait sujet aux déconnexions et ne garantissait pas l’achèvement avant l’échéance. Cette dualité résume un arbitrage fréquent en pratique : *débit* (CUDA/T4) contre *disponibilité et continuité* (MPS local).

4 Résultats

4.1 Reproduction principale et comparaison aux lignes de base

Le Tableau 2 (illustré par la Figure 2) compare la représentation DINO aux deux lignes de base de *même architecture*. La borne basse (initialisation aléatoire) situe le hasard ; la ligne de base supervisée fournit la comparaison « équitable » (même backbone, même protocole k -NN). On attend : aléatoire $\ll$ DINO, et DINO se rapprochant de — voire dépassant en k -NN — la version supervisée entraînée sur seulement 5 000 images étiquetées, illustrant l'intérêt d'exploiter 20 000 images *non* étiquetées.

TABLE 2 – Précision k -NN top-1 (%) sur le test STL-10, features gelées, $k = 20$. Chiffres à compléter depuis `results/main_results.json`.

Méthode (backbone ViT-Tiny/8)	k -NN top-1 (%)
Initialisation aléatoire (borne basse)	27,4
Supervisé (5 000 images étiquetées)	52,6
DINO (auto-supervisé, $\leq 24\,000$ img.)	37,2
<i>Pour mémoire</i> : supervisé, précision de classification directe	49,8

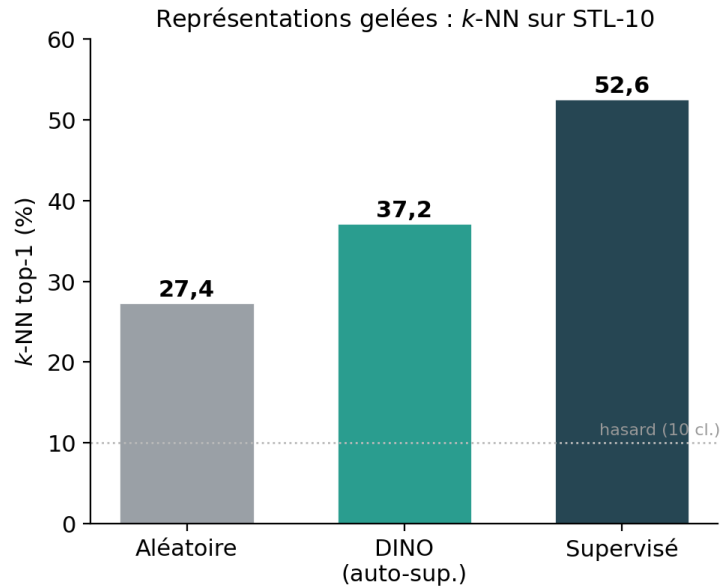


FIGURE 2 – Représentations gelées : précision k -NN sur STL-10 (mêmes chiffres que le Tableau 2). DINO (37,2 %) dépasse largement la borne aléatoire (27,4 %) mais reste sous la baseline supervisée de même architecture (52,6 %).

Remarque. Nous avons retenu le protocole **adaptatif** (§5b du notebook) plutôt que le pré-entraînement à durée fixe (§5) ; `figures/loss_curve.png` n'est donc pas produit. Le diagnostic de convergence (absence d'effondrement) est fourni par la courbe d'entraînement adaptatif (Figure 6), dont la précision k -NN sonde croît de façon monotone.

4.2 Résultat qualitatif : attention émergente

La Figure 3 montre les cartes d'auto-attention du token [CLS] (dernier bloc). Conformément au papier (Figures 1 et 3), différentes têtes se spécialisent sur des régions sémantiques distinctes

de l’objet, *sans aucune supervision*. C’est la propriété émergente phare de DINO ; sa présence à échelle réduite valide qualitativement notre reproduction.



FIGURE 3 – Cartes d’auto-attention par tête (token [CLS], dernier bloc) sur deux images de test STL-10 (figures/attention_dog.png, attention_horse.png). Différentes têtes se focalisent sur des régions distinctes de l’objet, *sans aucune supervision*.

5 Ablation et extension

Nous menons deux ablations exécutées inspirées de la Table 7 du papier (plus une troisième, décrite mais non exécutée), adaptées à notre cadre réduit (20 époques chacune). Chaque ablation ré-initialise un modèle et ne modifie *qu’un* composant. Le Tableau 3 et la Figure 4 regroupent les résultats.

A1 — Sans encodeur à momentum. *Changement* : l’enseignant devient une copie directe de l’étudiant à chaque pas (au lieu de l’EMA). *Hypothèse* : la cible n’est plus stable, le modèle *s’effondre* et la précision k -NN chute au niveau du hasard. C’est le composant le plus critique du papier (Table 7 : retrait du momentum $\rightarrow$ 0,1 %). *Résultat* : **19,6 %** — effondrement confirmé. La précision tombe *sous* la borne aléatoire (27,4 %, Tab. 2) et la perte de pré-entraînement *diverge* (5,5 $\rightarrow$ 8,4), double signature de l’effondrement. Le papier rapporte 0,1 % (le hasard sur 1000 classes) ; à 10 classes, où l’initialisation aléatoire atteint déjà 27,4 %, l’effondrement se lit comme un modèle *pire* que l’aléatoire — l’entraînement dégénéré a détruit la structure pourtant présente à l’initialisation.

A2 — Sans multi-crop. *Changement* : on retire les vues locales (2 vues globales seulement). *Hypothèse* : perte de l’objectif « local-to-global », baisse modérée de ~ 3 –5 points. *Résultat* : **31,6 %**, soit $-5,6$ points par rapport à DINO complet (37,2 %). Le papier mesure $-4,9$ points (72,8 $\rightarrow$ 67,9, Table 7) : nos deux estimations de la contribution du multi-crop concordent étroitement *malgré* une échelle $50\times$ plus petite, illustrant que l’*effet relatif* d’un composant se transpose mieux que la performance absolue. La variante reste stable (enseignant EMA conservé), d’où l’absence d’effondrement contrairement à A1.

A3 — Balayage de la température de l’enseignant τ_t . *Changement* : $\tau_t \in \{0,04, 0,07, 0,10\}$. *Hypothèse* : une température trop haute sous-affûte les cibles et rapproche de l’effondrement (centrage non compensé), tandis qu’une température basse maintient des cibles piquées. C’est une mise en évidence directe de l’équilibre centrage/affûtage. *Statut* : faute de budget de calcul (chaque valeur de τ_t exige un entraînement complet, soit ~ 5 h supplémentaires sur le GPU MPS local),

ce balayage n’a *pas* été exécuté. Nous en conservons l’hypothèse, directement vérifiable en relançant la cellule correspondante du notebook ; la Figure 7 du papier (étude de l’effondrement par entropie/divergence KL) corrobore l’effet attendu de τ_t .

TABLE 3 – Résumé des ablations (précision k -NN top-1 %, test STL-10). Source : `results/all_results.json`.

ID	Variante	k -NN top-1 (%)
—	DINO complet (référence)	37,2
A1	Sans momentum (copie directe)	19,6
A2	Sans multi-crop (2 globales)	31,6
A3	Balayage τ_t (0,04 / 0,07 / 0,10)	<i>non réalisé</i>

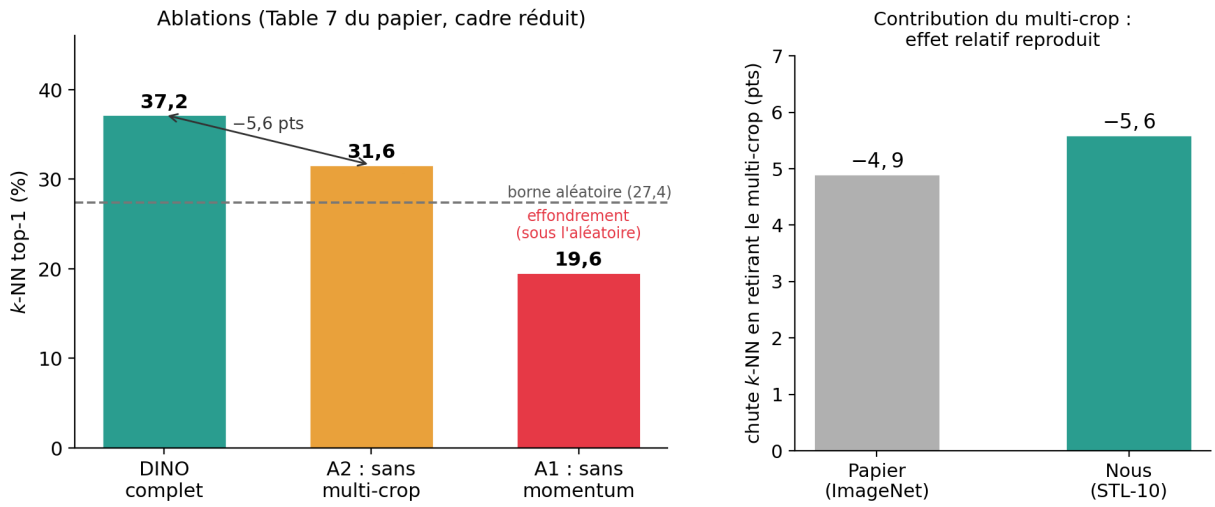


FIGURE 4 – **Gauche** : ablations en cadre réduit. Retirer le momentum (A1) effondre la représentation *sous* la borne aléatoire ; retirer le multi-crop (A2) coûte $-5,6$ points. **Droite** : la contribution du multi-crop que nous mesurons ($-5,6$ pt) coïncide presque avec celle du papier ($-4,9$ pt, Table 7), bien que l’échelle diffère d’un facteur ~ 50 .

5.1 Extension personnelle : entraînement adaptatif piloté par la métrique aval

Plutôt que de fixer *a priori* la taille du jeu non étiqueté — un choix arbitraire qui influe fortement sur le résultat — nous proposons un protocole d’entraînement **adaptatif**. Le pool d’images non étiquetées croît par tranches de **CHUNK** (4 000) ; après chaque tranche, on entraîne quelques époques puis on évalue un k -NN sonde rapide sur un petit échantillon tenu à l’écart (1 000 réf./1 000 requêtes). L’entraînement s’**arrête automatiquement** lorsque le gain de précision sonde passe sous un seuil (**min_delta**) pendant **patience** phases consécutives, ou atteint une cible. Le critère « combien de données suffisent ? » devient ainsi *empirique* : on lit directement sur la courbe (Figure 6) le point de saturation, et le meilleur modèle rencontré est conservé. La Figure 5 schématise cette boucle.

Motivation et hypothèse : le SSL devrait bénéficier de plus de données jusqu’à un plateau ; identifier ce plateau évite de gaspiller du calcul et justifie la taille retenue. *Limite* assumée : chaque phase ajoute simultanément des données *et* des pas d’optimisation, si bien que la courbe mêle effet-données et effet-durée ; nous l’interprétons donc comme un critère d’arrêt opérationnel, pas comme une loi d’échelle pure.

Extension : entraînement adaptatif piloté par la métrique aval

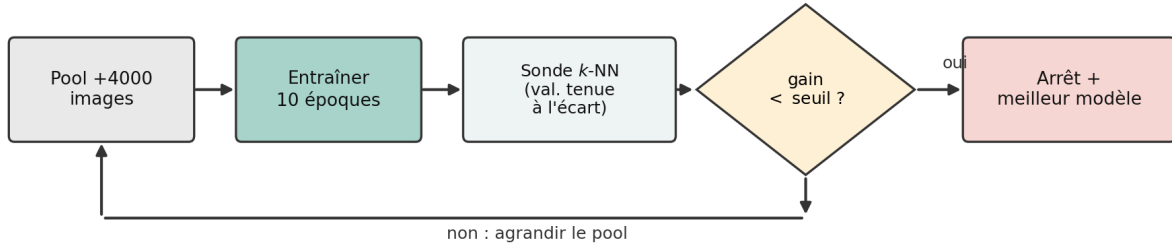


FIGURE 5 – Boucle d’entraînement adaptatif : le pool non étiqueté croît par tranches ; on entraîne, puis on sonde un k -NN sur une validation tenue à l’écart ; l’arrêt se déclenche dès que le gain passe sous un seuil. La quantité de données « suffisante » devient ainsi empirique plutôt qu’arbitraire.

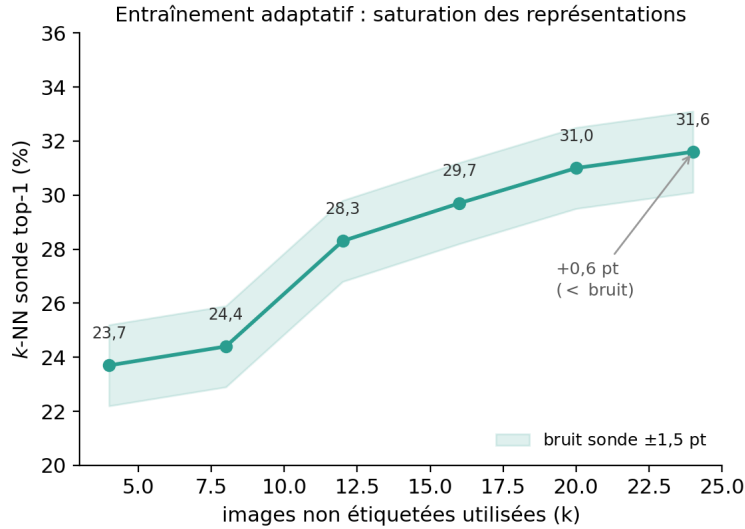


FIGURE 6 – Entraînement adaptatif : précision k -NN sonde en fonction du nombre d’images non étiquetées utilisées ; la bande grisée indique le bruit d’échantillonnage de la sonde ($\pm 1,5$ pt sur 1 000 requêtes). La précision croît de 23,7 % (4 000 img.) à 31,6 % (24 000 img.), avec un gain marginal décroissant sur la dernière tranche (31,0 $\rightarrow$ 31,6 %) signalant la saturation. Arrêt à 24 000 images / 60 époques, meilleur k -NN sonde 31,6 %.

6 Discussion critique

Ce qui fonctionne. DINO (37,2 %) dépasse nettement la borne aléatoire (27,4 %, soit +9,8 points), confirmant que la self-distillation apprend des représentations utiles *sans aucune étiquette* à échelle réduite. L’ablation A1 produit un effondrement franc (19,6 %, sous l’aléatoire, perte divergente), validant le rôle *load-bearing* de l’enseignant EMA — exactement la conclusion de la Table 7 du papier (retrait du momentum $\rightarrow$ 0,1 %). Surtout, A2 chiffre la contribution du multi-crop à $-5,6$ points, très proche des $-4,9$ points du papier : l’*effet relatif* d’un composant se reproduit fidèlement même quand les valeurs absolues, elles, ne sont pas comparables. Enfin, des cartes d’attention sémantiques émergent même avec un ViT-Tiny peu entraîné (Figure 3).

Ce qui fonctionne mal ou reste incertain. Contrairement au papier, où le k -NN DINO rivalise avec les features supervisées, notre DINO (37,2 %) reste *en deçà* du supervisé (52,6 %). C’est attendu et instructif : (i) 24 000 images et ~ 60 époques sont très en deçà du régime où le SSL brille (le papier utilise 1,28 M d’images, 300 époques) ; (ii) $K = 2048$ et un ViT-Tiny limitent la capacité ; (iii) le k -NN à un seul k et une seule graine a une variance non mesurée.

La courbe adaptative (Figure 6) étaye cette lecture : la précision sonde *croît encore* à 24 000 images ($23,7 \rightarrow 31,6\%$) sans saturer pleinement — l'écart au supervisé est donc une limite de *données/calcul*, non un défaut de la méthode, et se réduirait à plus grande échelle, en cohérence avec le papier.

Différences avec le papier et limites. Nos chiffres absolus ne sont pas comparables à ceux du papier (dataset, résolution, durée, taille de K et de batch tous réduits). Les limites principales : (i) *budget de calcul* — une seule graine, pas d'intervalles de confiance ; (ii) *évaluation* — k -NN uniquement, pas de *linear probing* ; (iii) *échelle* — le régime de données est trop petit pour que l'avantage du SSL sur le supervisé se manifeste pleinement. Des extensions naturelles (hors budget ici) seraient : augmenter le sous-échantillon et les époques (`FAST_MODE=False`), ajouter le *linear probing*, et moyenner sur plusieurs graines.

Conclusion. Cette reproduction réduite isole et teste l'idée scientifique de DINO — la self-distillation sans étiquettes avec enseignant EMA, centrage et affûtage — montre comment elle se transpose à un cadre Colab/STL-10, et identifie par ablation quels composants modifient les résultats et pourquoi. Conformément à l'esprit du projet, nous privilégions une analyse honnête des résultats (y compris négatifs) à la course à la performance.

Références

- [1] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. *Emerging Properties in Self-Supervised Vision Transformers*. ICCV 2021. arXiv :2104.14294.
- [2] A. Dosovitskiy *et al.* *An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale*. ICLR 2021.
- [3] M. Oquab *et al.* *DINOv2 : Learning Robust Visual Features without Supervision*. arXiv :2304.07193, 2023.